

Remote Team Time-Tracking & Project Approver

Problem Statement #46 (Developer Tools & IT Operations). Actors: Remote Developer, Engineering Manager, Jira System (external).

Requirements Table

Req ID	Type	Description	Priority	Acceptance Criteria	Rationale
FR-001	Functional	The system shall allow developers to log time against specific project tasks and prevent timesheet submissions exceeding 24 hours in a single calendar day.	High	Pass – a timesheet with 24 hrs or less per day goes through and lands in the manager's queue. Fail – someone manages to log 30 hours in one day and it still saves.	This one's kind of the whole point of the app. If people can't log their hours properly, nothing downstream (approvals, budget tracking) works either.
FR-002	Functional	The system shall let an Engineering Manager approve or reject a submitted timesheet, and if it's rejected, the manager must add a comment before it can be saved.	High	Pass – manager types a reason and the rejection goes through, developer gets pinged. Fail – manager rejects with the comment box left blank and it still saves.	Nobody likes getting a timesheet bounced back with no explanation. A required comment saves everyone a round of back-and-forth messages.
FR-003	Functional	The system shall pull the developer's task list from Jira so time can only be logged against tickets that are actually assigned to them.	Medium	Pass – dropdown only shows tickets assigned to the logged-in dev, synced within the last 15 min. Fail – a ticket that isn't theirs still shows up as selectable.	Otherwise there's nothing stopping someone from logging hours against random tickets, which basically makes the whole tracking system meaningless.
FR-004	Functional	The system shall work out each project's budget burn rate automatically, calculated from approved hours multiplied by each developer's hourly cost.	High	Pass – 100 approved hours at \$50/hr shows up as \$5,000 spent on the dashboard. Fail – the number shown is off from the manual calculation by more than 1%.	This is probably the feature managers actually care about most – nobody wants to find out a project blew its budget after the fact.
FR-005	Functional	The system shall notify the Engineering Manager automatically once a project's burn rate goes past 90% of what was budgeted.	Medium	Pass – crossing the 90% mark triggers a notification within about 5 minutes. Fail – the threshold is crossed and nothing gets sent.	Better to flag it early so a manager can actually do something, instead of finding out once the budget's already gone.

Req ID	Type	Description	Priority	Acceptance Criteria	Rationale
NFR-001	Nonfunctional (Performance & Security)	Timesheet approval status transitions and project budget burn charts must update across dashboards within 500 ms.	High	Pass – load testing shows updates consistently landing under 500 ms, and connections are confirmed to be running TLS 1.2 or above. Fail – any measured update takes longer, or an unencrypted connection gets through.	People are checking these dashboards from different time zones and expect what they see to actually be current, not stale by a few minutes.
NFR-002	Nonfunctional (Reliability / Availability)	The system shall stay available at least 99.5% of the time each month, not counting maintenance windows that were announced ahead of time.	High	Pass – monthly uptime logs come out at 99.5% or better outside of scheduled maintenance. Fail – logs show more downtime than that with no maintenance notice given.	Since the team is fully remote and spread across time zones, there isn't really a “off hours” window where downtime is fine.

FR-001 and NFR-001 (shaded rows) are the sample requirements given in the problem statement, kept here to round out the required five FRs and two NFRs.